Transaction Management & Concurrency Control:-

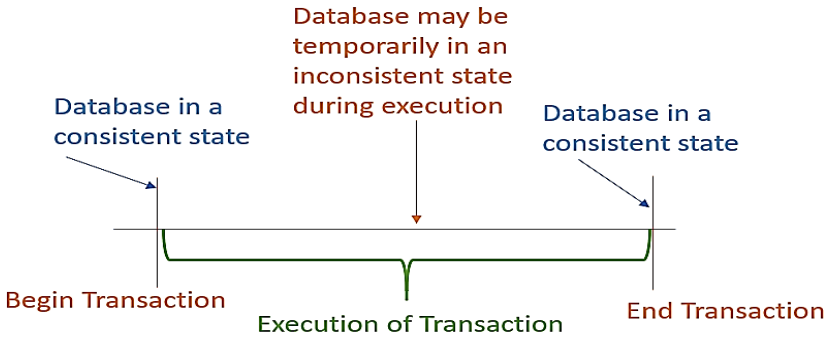
📣 Introduction of Transaction
- ACID properties
- transaction states
- scheduling
- conflict and view serializability

📣 Introduction to Concurrency Control
- Problems of concurrency control
- log based protocols
- Timestamp based protocol
- Deadlock
- Deadlock handling methods.

****

📣 **Introduction of Transaction concept**

A transaction is a set of operations perform a logical unit of work. Transaction is done by a single user or application program, which reads or updates set of contents of database.



A transaction consist Read and Write operations, which are held between the 'BEGIN TRANSACTION' and 'END TRANSACTION' statements. Database must be in a consistent state before and after execution of the transaction. Database is committed when state is changed from one consistent state to another. It must manage concurrent execution of transactions to avoid inconsistency.

Example:

When we go to withdraw money from ATM, we are doing the following set of operations:

1. Transaction Started
2. You have to insert an ATM card
3. Select your choice of language
4. Select whether savings or current account
5. Enter the amount to withdraw
6. Entering your ATM pin
7. Transaction processes
8. You collect the cash
9. You press finish to end transaction

A Transaction in DBMS there are three major operations are used

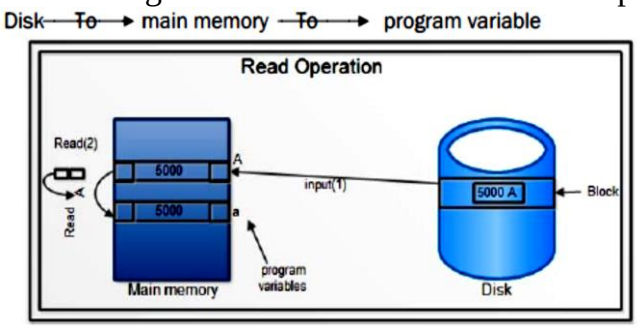
1. Read/ Access Data
2. Write/ Change Data
3. Commit

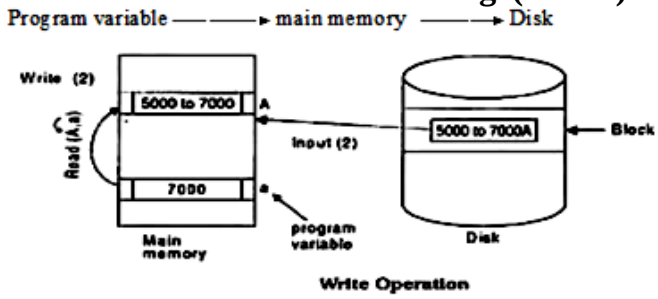
⊙ **Read Operation**

Read operation is used to read the value from the database, first brought into Main memory from disk and then its value copied into a program variable. A read operation does not change the image of the database in anyway.

- An image existed before the transaction occurred is called the **Before Image(BFIM)**

- An image exists after the transaction completed occurred is called the **After Image(AFIM)**



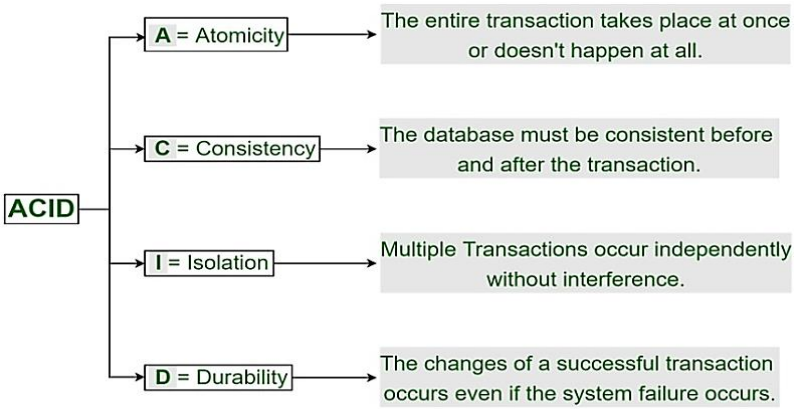


⊙ Write Operation

Write operation is used to write the value back to the database from the main memory. Whether write operation performed with the purpose of inserting, updating or deleting data from the database then changes the image of database.

📣 Transaction Properties (ACID)(**A**tomicity, **C**onsistency, **I**solation, and **D**urability):

These are used to maintain consistency in a database before and after the transaction. To ensure that a database should remain stable state after the transaction is executed.



1) Atomicity: (all action or nothing any action)

A transaction is a single unit of operations always executes all actions in one step or not executes any actions at all. When performing operations on a transaction, the operation should be completed totally rather than partially. If any of the operations aren't completed fully, the transaction gets aborted. Atomicity is also known as the 'All or nothing rule'.

Example: Let's assume that following **transaction T** consisting of **T1** and **T2**. **Account A**=Rs500 and **Account B**=200. **Account A** transfer Rs 100 to **Account B**.

Before Transaction Balance Account A=500 Before Transaction Balance Account B=200	
Transaction T	
T1	**T2**
Read(A); **A:=A-100;** **Write(A);**	**Read(B);** **B:=B+100;** **Write(B);**

After successful completion of the transaction, **Account A** consists of Rs400 and **Account B** consists of Rs300. If transaction fails after completion of **write (A)** but before **write (B)**, then amount has been deducted from **Account A** but not added to **Account B**. thus the value of **Account A** is Rs.400 and **Account B** is Rs.200 respectively. We lost Rs.100 because of transaction failure. "This shows the inconsistent database state because transaction is partially completed". A database is in consistent state, if transaction executed entirely or completely.

In order to maintain atomicity of transaction and database in consistent state, if transaction does not complete its execution or failure that time database returned its original values. System always keeps old values for restore when transaction never executed.

Example: one person is trying to book a ticket. They were able to select their seat and reach the payment gateway. But, due to bank server issues, they payment could not go through. A complete transaction means reserving your seat and paying for it as well. If any of the steps fail, the

operation will be aborted and you will be brought back to the initial state.

2) Consistency: (No violation of integrity constraints)

This property ensures that integrity constraints are maintained. It ensures that the database remains consistent before and after the transaction. To preserve the consistency of database, the execution of transaction should take place in isolation (no other transaction run concurrently when there is a transaction already running). If execution of transaction is successful, then the database remains in a consistent state. If the transaction fails, then the transaction will be rolled back and database will be restored to a consistent state. The transaction is used to transform the database from one consistent state to another consistent state. It is the responsibility of application programmer to ensure consistency of database.

Example: suppose that both T1 and T2 perform concurrently, then total amount before and after the transaction must be maintained.

Sum Before Transaction (500+200)=700	
Transaction T	
T1	**T2**
Read(A); **A:=A-100;** **Write(A);**	**Read(B);** **B:=B+100;** **Write(B);**

Here sum of (A, B) should be same before transaction and after transaction then the database is consistent. But In the case when T1 is completed but T2 fails, then sum after transaction is 600. Therefore database is not consistent. Inconsistency occurs because $\mathbf{T_1}$ is incomplete.

Example: one person is trying to book a ticket. They are able to reserve their seat but their payment hasn't gone through due to bank issues. Their transaction is rolled back but this isn't sufficient. This will be verify that the total sum of seats left in the train+sum of seats booked by users=total number of seats present in the train. After each transaction, consistency is checked to ensure nothing has gone wrong.

3) Isolation: (concurrent changes invisibles)

This property ensures that multiple transactions can run concurrently without leading to inconsistency of database state. Transactions execute separately without interference. i.e. transaction will be hidden from outside the transaction until the transaction terminate. When two or more transactions occur at the same time, consistency should be maintained. Any modifications made in one transaction will not be visible to other transactions until the change is committed to the memory. The concurrency control of the DBMS required the isolation property.

Transaction T	
T1	**T2**
Read(A);=50 **A:=A*100; =5000** **Write(A); =5000** **Read(B); =100** **B:=B-50; =50** **Write(B); =50**	**Read(A); =5000** **Read(B); =100** **C:=A+B; =5000+100** **Write(C); =5100**

Suppose **T1** has been executed till Read (B) and then **T2** starts. Interleaving of operations takes place due to which **T2** reads correct value of **A** but incorrect value of **B** and inconsistent with the sum at end of transaction **T2**: (A+B)=(5000+100)=5100)

But after the successful transaction the sum will be: **T2:** (A+B)=(5000+50)=5050).

This results in database inconsistency, due to a loss of Rs. 50. if the transaction T1 is being executed and using the data item A and B, then that time data item can't be accessed by any other transaction T2 until the transaction T1 will finish.

Example: 2 people trying to book the same seat. Two transactions are happening at the same item on the same database, but in isolation. To ensure that one transaction doesn't affect the other, they are serialized by the system. This is done so as to maintain the data in a consistent state. Two people that click 'Book Now', do so with a gap of a few seconds. In that case, the first person's transaction

goes through and he receives their ticket as well. When the second person clicks on 'Book Now' and is redirected to the payment gateway, since the first person's seat has already been booked, it will show an error notifying the user that no seats are left on the train.

4) Durability: (Permanency)

All the above three properties should be satisfied while the transaction in progress. But durability can happen after the completion of transaction. Once a transaction successfully completes or committed, these changes are immediately written into permanent storage (disk) and database reaches a consistent state. That consistent state cannot be lost, even system's failure or crash. It is the responsibility of recovery manager to ensure durability in the database.

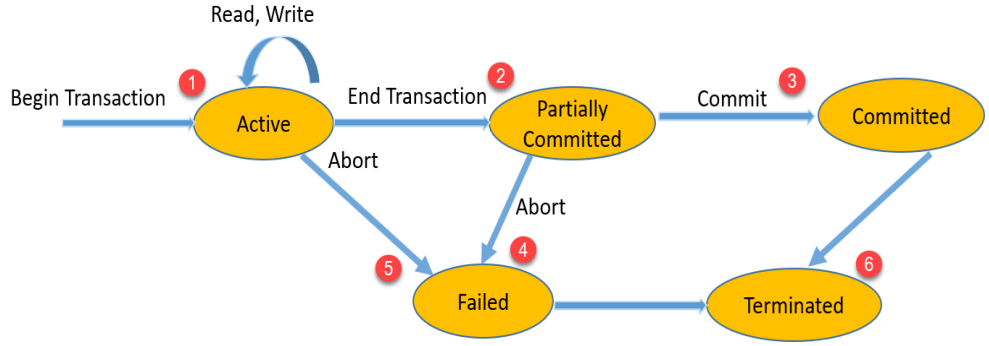
Example: Let, Ti is a transaction that transfers Rs 100 from account A to account B.

T1	T2
Read(A); **A:=A-100;** **Write(A);** **Commit;**	**Read(B);** **B:=B+100;** **Write(B);** **Commit;**

After write (A) operation system failure then data is inconsistent, hence system is not stable. But system failure after commit operation then system is (consistent) stable.

📣 Transaction States

A transaction goes through different states throughout its life cycle.



- **Active State:** The active state is the first state of every transaction. In this state, the transaction is being executed. Example: Insertion or deletion or updating record is done here. But all the records are still not saved to the database.
- **Partially committed:** A transaction goes into the partially committed state after executes its read and write operation, but the data is still not saved to the database. Example: calculating total marks, final display the total marks step is executed in this state.
- **Failed:** If any problem is detected either during active state or partially committed state than transaction enter in the failed state. Example: total mark calculation, if the database is not able fire a query to fetch the marks.
- **Aborted:** If a transaction is failure, then database recovery system will make sure that database is in consistent state. If not, it brings the database to consistent state by aborting or rolling back the transaction. If transaction failure in the middle of the transaction, all the executed transactions are rolled back to it consistent state. Once the transaction is aborted. it is restarted to execute again. The database reaches the BFIM.
- **Committed:** transaction is in a committed state if it executes all its operations successfully. In this state, all the effects are now permanently saved on the database system.
- **Terminated:** Either committed or aborted, the transaction finally reaches this state.

➢ Concurrent Execution

Transaction systems allow multiple transactions to run concurrently i.e. executing more than one

transaction at the same time. Interleaved execution technique is used for concurrent execution. Concurrent execution of multiple transactions leads several difficulties with consistency of data.
Example: Suppose that two ticket agents are dealing with the seat booking system of a flight. If both agents book last available seat in the flight, it would result in the overbooking or double booking of that seat. This leaves the database in an inconsistent state.

To maintain consistency in concurrent execution of transactions requires run **serially** i.e**.** one at a time, each transaction starting only after the previous one has completed.

Transaction T	
T1	**T2**
Read(A); **Write(A) ;** **Read(C);** **Write(C) ;**	 **Read(B);** **Write(B);**

Firstly, T1 executes then T2 and then again T1. This is **interleaving**. This way multiple transactions are run concurrently in the system.

⊙ **Advantages of concurrent execution of transactions:**

1. Increased CPU utilization
2. Increase system throughput
3. Better average response time and turnaround time
4. Better disk utilization
5. Better average waiting time

📣 **Scheduling (Represents order or execution of transactions)**

- When multiple transactions are executing concurrently and perform all operations in sequential order, this process is known as a schedule.
- A bundle of transactions executing together is called a schedule.

So, if both these two transactions are carried away simultaneously, it will be called a schedule. The Schedule will decide and order in which the instructions are to be executed.

Transaction T2 is follows by T1. (T1 -----→ T2).

T_1	T_2
R(A)	R(B)
W(A)	W(B)
R(B)	R(A)
W(B)	W(A)

S_1

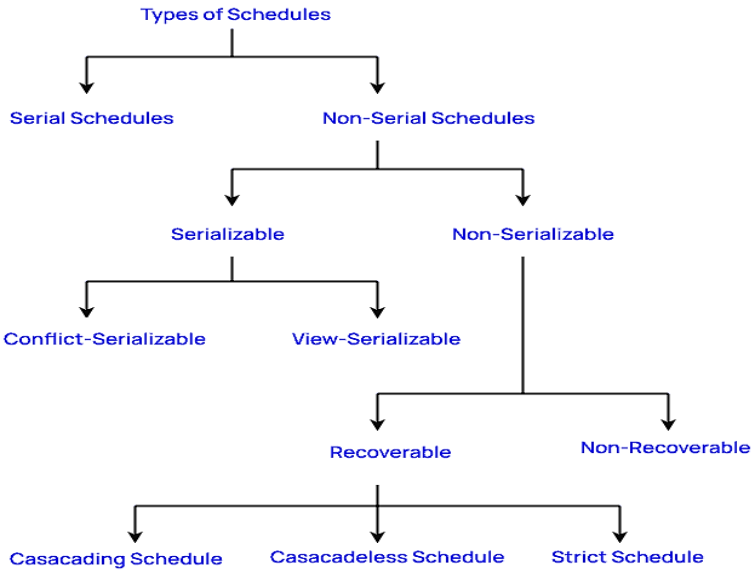
T_1	T_2
R(A) W(A)	
	R(B) W(B)
R(B) W(B)	
	R(A) W(A)

S_2

T_1	T_2
R(A) W(A) R(B) W(B)	
	R(B) W(B) R(A) W(A)

In above schedules have different orders to execute the instructions. Concurrent transactions that preserves the execution order of operations in each transactions.
All the operations of transaction **T1** are executed before the operations of transaction **T2.** This is not always necessary.

⊙ **Types of schedules:**



1) Serial Schedule -

All the transactions are executed serially one after another. In serial schedule, a transaction does not start execution until the currently running transaction finishes execution. This type of execution of transaction is also known as non-interleaved execution. Serial Schedule are always recoverable, cascadeless, strict and consistent. A serial schedule always gives the correct result. **Example:**

Initial Value of A=100

T_1	T_2
Read (A); A=A + 50; Write (A)	
	Read (A); A=A + 100; Write (A)

After Transaction of T1 & T2 i.e. $T_1 \longrightarrow T_2$ is equal to $T_2 \longrightarrow T_1$

T_1	T_2
A=100; 150=100 + 50; A=150	
	A=150; 250=150 +100; A=250

T_2	T_1
A=100; 150=100 + 100; A=200	
	A=200; 250=200 +50; A=250

- There are two transactions T1 and T2 executing serially one after another. Transaction T1 executes first. After T1 completes its execution, transaction T2executes.
- **Advantage -** it always guarantee a consistent state.
- **Disadvantage -** there is no concurrent execution of transactions.

2) Non-Serial Schedule

Operations of multiple transactions are interleaved. This might lead to concurrency problem. The other transaction proceeds without waiting for the previous transaction to complete. i.e. concurrent transactions are overlapping .The transactions are executed in a non-serial manner but keeping the result correct same as the serial schedule.

Transaction T	
T1	**T2**
Read(A); **A:=A-10;** **Write(A) ;**	
	Read(A); **A:=A*10;** **Write(A);**
Read(B); **B:=B+100;** **Write(B);**	

Transaction T	
T1	**T2**
Read(A); **A:=A-10;** **Write(A) ;** **Read(B);** **B:=B-10;** **Write(B) ;**	 **Read(A);** **A:=A+100;** **Write(A);** **Read(B);** **B:=B+100;** **Write(B);**
Transaction T1 and T2 are interleaved. These transactions conflict. operations of T1 overlap with the operations of T2	

Transaction T1 & T2 are interleaved. These transactions do not conflict. This is a non-serial schedule.

A non-serial schedule that produces the same result as some serial schedule is called a serializable schedule. The Non-Serial Schedule can be divided into Serializable and Non-Serializable

- **Advantage-** concurrency of executing the transactions. The source utilization and execution time will be less and will have a faster response time.
- **Disadvantage -** a potential risk that the system may go into an inconsistent state

3) Complete Schedule –

A schedule that contains last operations of each transaction is an either commit or an abort are called complete schedules. **Example:** Consider following schedule with three transactions T_1, T_2 and T_3.

T_1	T_2	T_3
Read(A);		
	Write(A);	
Read(B);		
		Write(B);
Commit;		
	Commit;	
		Abort;

The last operation performed under every transaction is either "commit" or "abort".

4) Equivalent schedule

If the two transactions generate same result after their execution on database is called as equivalent Schedule. **Example:** the schchdule-3 is an equivalent schedule to schcdule-1 &schedule-2.

Schedule S_1	
T_1	T_2
A = A * 10; B = B - 10;	 A = A/10 ; B = B * 2;

Schedule S_2	
T_1	T_2
	A = A/10; B = B * 2;
A = A * 10; B = B - 10;	

Schedule S_3	
T_1	T_2
A = A * 10; B = B - 10;	 A = A/10; B = B * 2;

5) Recoverable Schedule –

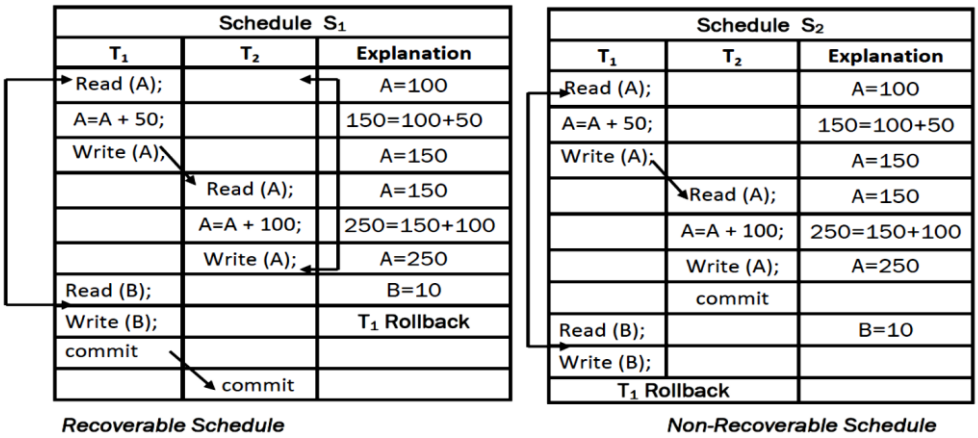
if a transaction is reading a value which has been updated by other transaction then this transaction can commit only after the commit of other transaction which is updating value.

Example – Consider the following schedule involving two transactions T_1 and T_2.

T_1	T_2
Read(A);	
Write(A);	
	Write(A);
	Read(A);
Commit;	
	Commit;

This is a recoverable schedule, **T_1commits** before **T_2**read values **T_2** correct.

Example:



Schedule S_1

T_1	T_2	Explanation
Read (A);		A=100
A=A + 50;		150=100+50
Write (A);		A=150
	Read (A);	A=150
	A=A + 100;	250=150+100
	Write (A);	A=250
Read (B);		B=10
Write (B);		T_1 Rollback
commit		
	commit	

Recoverable Schedule

Schedule S_2

T_1	T_2	Explanation
Read (A);		A=100
A=A + 50;		150=100+50
Write (A);		A=150
	Read (A);	A=150
	A=A + 100;	250=150+100
	Write (A);	A=250
	commit	
Read (B);		B=10
Write (B);		
T_1 Rollback		

Non-Recoverable Schedule

Schedule **S1** is recoverable schedule If Transaction **T1** failure after **Read (B)** operation then it has to be **rolled back T1** &**T2** also rollback because **T2** followed by **T1.** But in Schedule **S2** if **T1** transaction fails, it has to be roll backed. But **T2** is already committed, so it cannot be roll backed. Schedule **S2** is a Non recoverable Schedule.

6) Cascadeless Schedule:

If some transaction $\mathbf{T_j}$ wants to read value updated or written by some other transaction $\mathbf{T_i}$, then the commit of $\mathbf{T_j}$ must read it after the commit of $\mathbf{T_i}$. Cascadeless schedule allows only committed read operations. Therefore, it avoids cascading roll back and thus saves CPU time. **Example:** Consider the following schedule involving two transactions T1 and T2.

T_1	T_2
Read(A);	
Write(A);	
	Write(A);
Commit;	
	Read(A);
	Commit;

This schedule is Cascadeless. Since the updated value of **A** is read by T_2 only after the updating transaction i.e. T_1 commits.

Cascading Schedule: Failure of one transaction causes several other dependent transactions to rollback or abort, then such a schedule is called as a **Cascading Schedule** or **Cascading Rollback** or **Cascading Abort**. It simply leads to the wastage of CPU time.

7) Strict Schedule

A schedule is strict, if a value written by a transaction cannot be read or overwritten by another transaction until the transaction either aborted or committed. Intervening of uncommitted data is not allowed. If it satisfies the following conditions:

1. **T2** reads a **data item A** after **T1** has terminated (aborted or committed).
2. **T2** writes a **data item A** after **T1** has terminated (aborted or committed).

T_1	T_2
Read(A);	
Write(A);	
Commit;	
	Write(A);
	Read(A);
	Commit;

Every Strict schedule is both recoverable and Cascadeless Schedule.

8) Serializiable schedule

A Serializiable schedule always leaves the database in a consistent state. A serial schedule is always a Serializiable schedule. Only one transaction executes at a time and the other starts when the already running transaction finished. A non-serial schedule needs to be checked for Serializability.

If a given non-serial schedule of 'n' transactions is equivalent to some serial schedule of 'n' transactions, then it is called as a **Serializiable schedule.**

Time	T_1	T_2
T1	Read(A);	
T2	Write(A);	
T3	Commit;	
T4		Read(B);
T5		Write(B);
T6		Commit;
T7	Read(C);	
T8	Write(C);	
T9	Commit;	

Serial Schedules	Serializiable Schedules
No concurrency is allowed. All transactions execute serially one after the other.	Concurrency is allowed. Multiple transactions can execute concurrently.
Serial schedules lead to less resource utilization and CPU throughput.	Serializiable schedules improve both resource utilization and CPU throughput.
Serial Schedules are less efficient as compared to Serializiable schedules.	Serializiable Schedules are always better than serial schedules.

Serializability

When multiple transactions are running concurrently then there is a possibility that the database may be left in an inconsistent state. Serial execution of a set of transactions preserves database consistency. A schedule is Serializiable if it is equivalent to a serial schedule. A serializable schedule always database leaves in the consistent state.

When several concurrent transactions are trying to access the same data item but in serializability, the order of read and write operations are important.

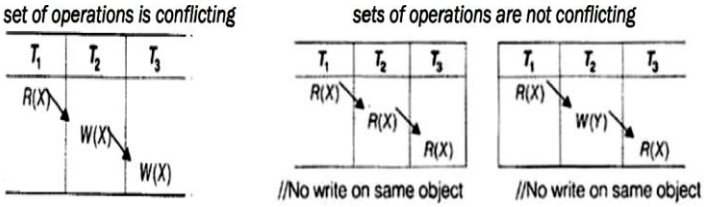
The main objective of serializability is to find non-serial schedules that allow transactions to execute concurrently without interfering and produce a database in consistent state i.e. Helps to identify which non-serial schedules are correct and will maintain the consistency of the database.

- **Conflicting Operations:** In a given schedule, Two operations are called as conflicting operations, if they satisfy all the following three conditions:
  - Both the operations belong to different transactions
  - Both the operations are on the same data item
  - At least one of the two operations is a write operation

When two or more transactions in a non-serial schedule execute concurrently, then there may be some conflicting operations.

- Ti read (A) and Tj read (A) = Don't conflict,
- Ti read (A) and TjWrite (B) = Don't conflict,
- Ti read (A) and Tj Write (A) = Conflict,
- Ti Write (A) andTj read (A) = Conflict,
- Ti Write (A) andTj Write (A) = Conflict.

Possible conflict: write-write, write-read, read-write.

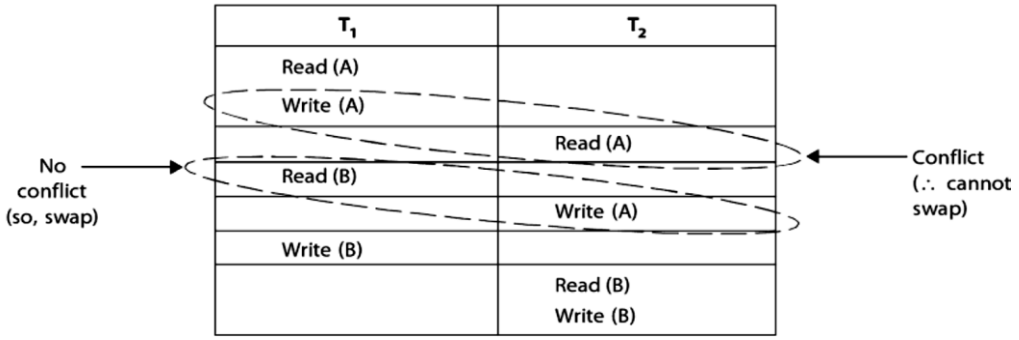


Trans Ti / Tj	Read	Write
Read	Don't conflict	Conflict
Write	Conflict	Conflict

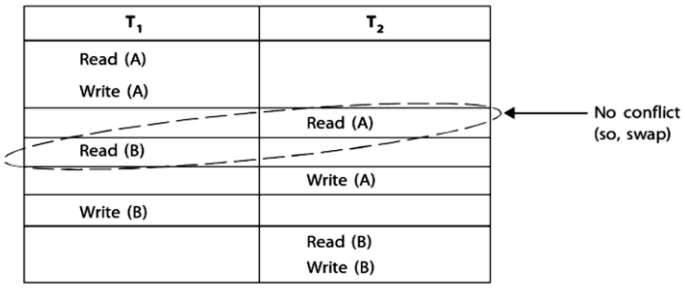
Example: Consider the following schedule:

T_1	T_2
Read (A) Write (A)	
	Read A
Read (B)	
	Write (A)
Write (B)	
	Read (B) Write (B)

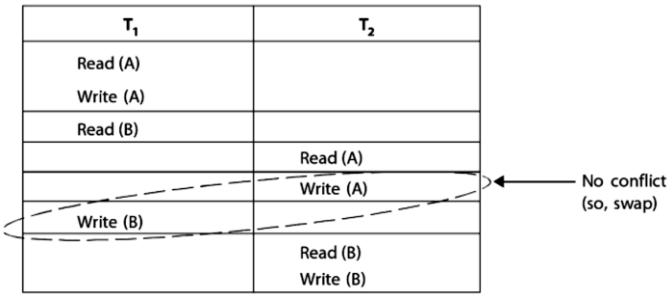
Are there any conflicting operations in T1or T2 Find out a serial schedule for it? Make it conflict Serializiable?



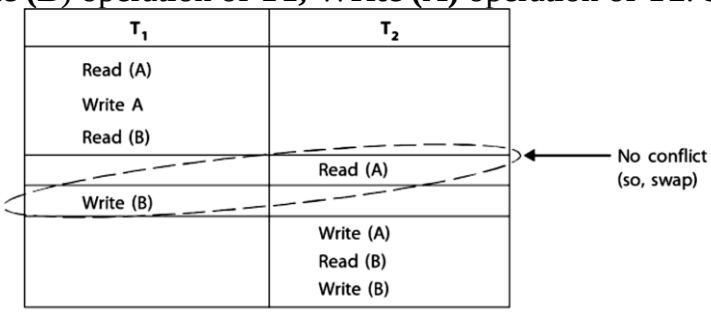
- **Write (A)** operation of **T1** conflicts with the **Read (A)** operation of **T2** because both of transaction perform operation on the same data-item **(A).**
- **Write (A)** operation of **T2** does not conflict with **Read (B)** operation of **T1** because both transaction access different data items.
- **Write (A)** operation of **T2** does not conflict with **Read (B)** operation of **T1.** But we can swap these operation generate an equivalent schedule**.**



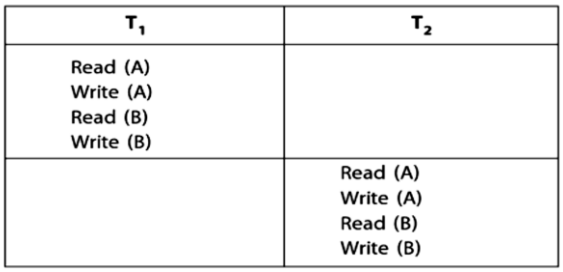
We can swap the **Read (B**) operation of **T1, Read (A)** operation of **T2**. So, we get:



We can swap the **Write (B**) operation of **T1, Write (A)** operation of **T2**. So, we get:



We can swap the **Write (B**) operation of **T1, Read (A)** operation of **T2**. So, we get:



After swapping final schedule is a serial schedule. A non-serial schedule S is Serializable is equivalent to a serial schedule. **There are two types of Serializable schedule:**

1. **Conflict Serializable schedule**
2. **View Serializable schedule**

1) Conflict Serializable Schedule:

If a given non-serial schedule can be converted into a serial schedule by swapping non-conflicting operations, then it is called as a **conflict Serializable schedule**.

Conflict Equivalent

if a schedule **S1**can be transformed into a schedule **S2** by a series of swapping of non-conflicting operation, then these **S1and S2** are **conflict equivalent**. If schedule **S1** is **conflict serializable** if it is conflict equivalent to a serial schedule. Two schedules are said to be conflict Equivalent if one schedule can be converted into other schedule after swapping non-conflicting operations.

Example: S2 is conflict equivalent to S1 (S1 can be converted to S2 by swapping non-conflicting operations).Two schedules are said to be conflict equivalent if and only if:

1. Both schedule contain same set of the transaction (ordering of operation within each transaction)
2. If each pair of conflict operations ordered in S_1 and $S_2$are same.

Non-serial schedule

T1	T2
Read(A)	
Write(A)	
	Read(A)
	Write(A)
Read(B)	
Write(B)	
	Read(B)
	Write(B)

Schedule S1

Serial Schedule

T1	T2
Read(A)	
Write(A)	
Read(B)	
Write(B)	
	Read(A)
	Write(A)
	Read(B)
	Write(B)

Schedule S2

Schedule S1 can be transformed into Schedule S2. Schedule S2 is a serial schedule where Transaction T2 follows Transaction T1, by series of swaps of non-conflicting instructions. Therefore **Schedule S1 is conflict serializable.**

Example of a schedule that is not conflict serializable:

T_3	T_4
Read(A);	
	Write (A);
Write (A);	

We are unable to swap instructions in the above schedule to obtain either the serial schedule< T3, T4 >, or the serial schedule < T4, T3 >

- **Testing for Conflict Serializability of a Schedule:**

To test whether a particular schedule is conflict serializable or not using constructs a precedence graph (serialization graph) or directed graph. A precedence graph for a schedule S contains G=(V,E)

Where 'V' is a set of vertices and 'E' is a set of edges Notations:

Set of vertices consists of all the transactions participating in the schedule

Set of edges consists of all edges **Ti→Tj** for which one of the following

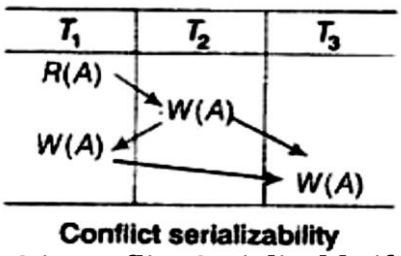
Three condition hold:

I. Ti executes Write (A) before Tj executes Read (A)

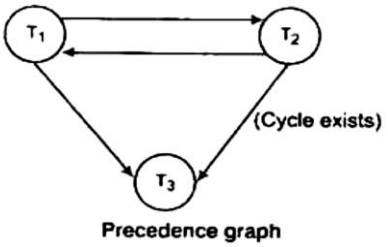
II. Ti executes Read (A) before Tj executes Write (A)

III. Ti executes Write (A) before Tj executes Write (A)

If an edge, Ti→Tj exists in the precedence graph then in any serial schedule,S_2 equivalent to S_1,Ti must appear before Tj. For example, schedule S, is



Conflict serializability



Precedence graph

A schedule S is conflict Serializable if and only if their precedence graph is acyclic.

Firstly T1 is executed thenT2, then again T1and then T2.

- If precedence graph has a cycle, then schedule S is not conflict Serializable.
- If precedence graph has no cycles, then schedule S is conflict Serializable

2) View Serializable Schedule: A schedule is view Serializable, if it is view equivalent to some serial schedule. Every Conflict Serializable is also View Serializable, but every view serializable schedule is not conflict Serializable it has **blind writes.**

View Equivalence: schedulesS1 andS2 are view equivalent, if they satisfy all the following conditions:

1. Initial Read ("Initial readers must be same for all the data items"):

An initial read (first read operation on data item **A** is called the initial read) of both schedules must be the same. Suppose two schedule **S1** and **S2**. In schedule **S1**, if a transaction **T1** is reading the **data item A**, then in **S2**, transaction **T1** should also **read A**.

Schedule S1 (Non Serial)	
T_1	T_2
Read(A);	
Write(A);	
	Read(A);
	Write(A);
Read(B);	
Write(B);	
	Read(B);
	Write (B);

S2 is the serial schedule of S1. They are view equivalent then schedule S1 is view serializable.

Schedule S2 (Serial)	
T_1	T_2
Read(A);	
Write(A);	
Read(B);	
Write(B);	
	Read(A);
	Write(A);
	Read(B);
	Write (B);

Above two schedules are view equivalent because Initial read operation in **S1** is done by **T1** and in **S2** it is also done by **T1**.

2. Updated Read ("Write-read sequence must be same"):

In **S1**, transaction **T2** reads value of **A**, updated by **T1**. In **S2**, the same transaction **T2** reads **A** after it is updated by **T1**.

In **S1**, reads value of **B**, updated by **T1**. In **S2**, Same transaction **T2** reads value of **B** after it is updated by **T1**.The update read condition is also satisfied for both the schedules.

3. Final Write ("Final writers must be same for all the data items") :

In schedule **S1**, Final write operation on data item **A** is done by transaction **T2**. In **S2** also transaction **T2** performs final write on data item **A**.

In schedule **S1**, final write operation on **B** is done by transaction **T2**. In schedule **S2**, final write on **B** is done by **T2**. Above two schedules is view equal because Final write operation on data items **A & B** in **S1** is read by **T2** and in **S2**, the final write operation is also read by **T2**.

Introduction to Concurrency Control

Executing more than one transaction at a time is called concurrent execution of transactions.

Concurrency Control: The technique is used to protect data when multiple users are accessing same data concurrently (same time) is called concurrency control.Concurrency control increases the throughput. It reduces waiting time of transaction.

When more than one transaction is running simultaneously there are chances of a conflict to leave database in an inconsistent state. To handle these conflicts we need concurrency control, which allows the transaction to run concurrently but maintaining the consistency of data by implementing Locks and Timestamps.

Locks and Timestamps protocols provide an environment in which concurrent transactions can preserve their consistency and isolation properties.

Purpose Concurrency control

- To implement Isolation- *one transaction cannot interleaving with other transaction*
- To preserve database consistency & integrity
- To resolve conflicts operations (read-write & write-write).

Three Problems of Concurrency Control

When concurrent transactions (i.e. accessing same data at same time) are executed in an uncontrolled manner then several problems can occur i.e. Uncontrolled manner means Execution of conflict pairs without any restriction.

Example: In a multi user environment system, multiple users can access the same database at a time. If the there is an BANK database and multiple users needs to perform the transactions without any restriction, then there may be the following concurrency problems can happens.

The concurrency control has 3-main problems

- Lost update problem
- Dirty read (uncommitted data) problem
- Unrepeatable read (inconsistent retrievals) problem

⊙ Lost update problem(write-write problem)

A lost update problem occurs when two transactions access same data item and their operations interleaved that makes value of some database item incorrect.

Schedule S_1		
T_1	T_2	Explanation
Read (A);		A=1000;
A=A – 50;		950=1000-50;
	Read (A);	A=1000;
	A= A + 100;	1100=1000+100;
Write (A);		A=950;
	Write (A);	**A=1000;** *Finally A store 1000*

Update Lost

i.e. if transactions **T1** and **T2** both read data item and then update it, but updation of **T1** will be overwritten by the **T2**". Both transaction are updating same data item and one of the updates is lost (overwritten by the other transaction).Finally database store value of data item A=1100 because Result of **T1** overwritten by **T2.** If **T1** lost update of value **A** data item.

⊙ Dirty read (uncommitted data) problem

A dirty read problem occurs when one transaction updates a database item but running transaction fails after some reason. The updated value is accessed by another transaction before it is changed back to the original value.

Schedule S_1		
T_1	T_2	Explanation
Read (A);		A=100;
A=A + 20;		120=100 + 20;
Write (A);		A=120;
	Read (A); *(T_2Read updated value of A)*	A=120;
	A= A + 10;	130=120+10;
	Write (A);	**A=130;** *instead of 110*
Read (B);		
	Commit;	**A=130;**

Power Failure & Rollback T_1

Consider a schedule S, transactions **T2** read the value of **A** updated by transaction **T1**. Since the transaction**T1** fails and rolls back after the Read (B) operation i.e. **T1** obtain the original value of **A** and cancel the updated value of **A**. But transactions **T2** process on updated value of **A**, then database goes into inconsistent state.

In a dirty read, data are not committed when two transactions are executed concurrently and first transaction **T1** is rolled back after the second transaction **T2** has already accessed the uncommitted data. Thus, it violates the isolation property of transactions.

⊙ Unrepeatable read (inconsistent retrievals or W-R Conflict)

In an unrepeatable read, the transactions **Ti** reads a data item value twice (double) then the data item value changed by other transaction **Tj** in between the read operation. Hence **Ti** reads different values for its two read operation on same data items. The new value will be inconsistent with the previous value."

Schedule S_1		
T_1	T_2	Explanation
Read (A);		A=1000;
	Read (A);	A=1000;
	A= A + 2000;	3000=1000+2000;
	Write (A);	**A=3000;**
Read (A);		**A=3000**

Consider an example of unrepeatable read in which of **T1** were to read the value of **A** after **T2** had

updated **A**, the result of **T1** would be different. i.e Inconsistency occurs because **T1** reads several values of data item **A** from database while **T2** update value some of them.

Lock Based Protocols –

A lock is a variable associated with data item to control concurrent access of data item. It provides a guaranteed access of data item from database to a current transaction. When a transaction is access data item, first to check the associated lock. If no other transaction holds the lock, then he locks data item. This protocol requires that all the data items must be accessed in a mutually exclusive manner i.e. if one transaction has access data item at a time and other transactions are locked.

- Lock is acquired when access to the data item
- Lock is released when the transaction is completed

Concurrency problems arise when multiple users try to update or insert data into a database table at the same time. Such concurrent updates can cause data to become corrupt or inconsistent. Locking is used to prevent such concurrent updates to data.

Example-In traffic, there are signals which indicate stop and go. When one signal is allowed to pass at a time, then other signals are locked.

✓ **Types of Locks:** Several types of locks are used in concurrency control.

- **Binary Locks**
- **Shared/Exclusive (R/W) Locks**
  - **Shared Lock (Lock-S) :** Read data item value
  - **Exclusive Locks (Lock-X):** Read/Write both operations.

1. **Shared Lock (S):** also known as Read-only lock. It is used when data item has to be read. When transaction holds a lock, then it can't update data item.
2. **Exclusive Lock (X):**It is used when data item has to update or could be read or write.

➢ **Binary Locks :**A binary lock can have two states or values: lock item and unlock item

- If Lock(A)=1 means **item A** cannot be accessed by transaction and it is forced to wait
- If Lock (A)=0 means **item A** can be accessed by transaction.
- Binary lock enforces mutual exclusion on the data item.

⊙ **Locking operations:**

A lock associated with an item A, there are three locking operations called read_lock(A), write_lock(A) and unlock(A) represented as (S-shared lock, X-exclusive lock)

- **Lock-S(A) or read-locked-**Multiple transactions are allowed to read operation on data item
- **Lock-X(A) or write-locked-**Single transaction holds exclusive lock on data item.
- **Unlock (A)**
- Any number of transactions can hold shared locks on an item, but if any transaction holds an exclusive on the item then no other transaction hold any lock on the item.

When we use shared/exclusive locking, then system must enforce following rules:

1) A transaction T must issue operation read_lock(A) or write_lock(A) before any read_item(A) or write_item(A) operation is performed in T.

2) Transactions T must issue the operation unlock(A) after all read(A) and write(A) operations are completed in T.

3) A transaction T will not issue a read_lock(A) or write_lock(A) operation if it already holds a read(shared)lock or a write (exclusive) lock on item A.

4) A transaction T will not issue unlock(A) operation without holding a read(shared) lock or a write (exclusive) lock on item A.

T_1	T_2
Lock-X(B); Read (B); B=B – 50; Write (B); Unlock (B); } Exclusive Lock	
	Lock-S (B); Read (B); Unlock (B); } Shared Lock

⊙ **Compatibility of Locks**

A transaction may be granted a lock on data item if the requested lock is compatible with locks already held on the item by other transactions. If a transaction **T1** requests a shared lock **Lock-S(Q)** on **item Q,** Currently transaction **T2** hold an exclusive lock **Lock-X(Q)** on **item Q**.**T1**request can't be granted lock.

- if two transactions only read the same data item they do not conflict
- But if one transaction writes a data item and another either read or write the same data item, then they conflict with each other.

	Shared Lock	Exclusive Lock
Shared Lock	True (Access allowed)	False (Access not allowed)
Exclusive Lock	False (Access not allowed)	False (Access not allowed)

T_1	T_2
Lock-X(P)	Lock-X(Q)
Lock-S(Q)	Lock-S(P)

To access a data item, transaction must first lock data item. If data item is already locked by another transaction in incompatible locks, the concurrency control manager will not grant the lock.

⊙ **Problem with Simple Locking:**

- Inconsistency of database
- Deadlock Occur
- No guarantee of Serializability

Schedule S1	
T_1	T_2
lock-X(B);	
Read(B);	
B:=B-50	
Write(B);	
	lock-S(A);
	Read(A);
	lock-S(B);
lock-X(A);	

- T2 is waiting for T1 to release its lock on B, while executing lock-X(A)causes T1 is waiting for T2 to release its lock on A, Such a situation is called a deadlock.
- To handle a deadlock of T1or T2, it must be rolled back and released its locks.
- Starvation is also possible if concurrency control manager is badly designed.
  - A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
  - The same transaction is repeatedly rolled back due to deadlocks.
- Concurrency control manager can be designed to prevent starvation.

⊙ **Conversion of locks**

A transaction **T** that already hold lack on **data item A** has allow in certain condition to convert the lock from one locked state to another state.

- Converting from shared lock to exclusive lock is called **upgrading lock.**
- Converting from exclusive lock to shared lock is called **downgrading lock.**

In two phase locking, upgrading in growing phase and downgrade in shrinking phase.

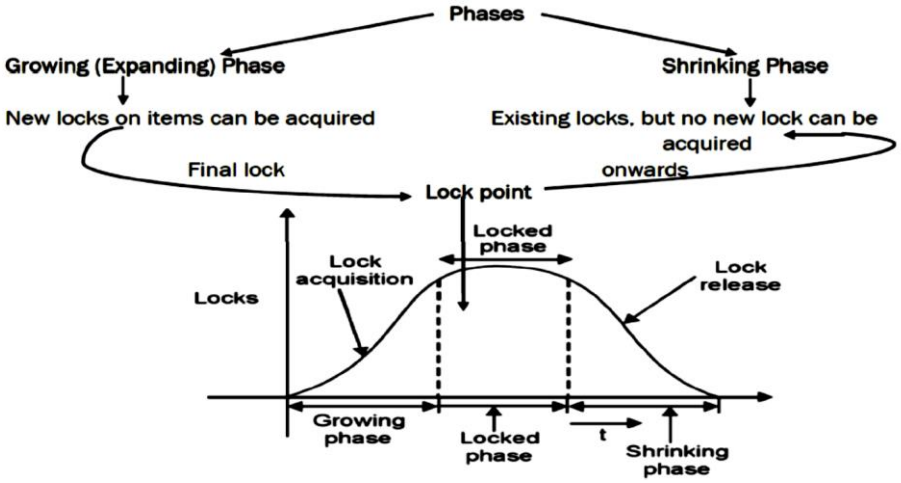
Two Phase Locking (2PL) Protocol:

The Two Phase Locking Protocol defines rules for how to acquire locks on a data item and how to release locks. Two phase locking (2PL) is a concurrency control method guarantees that serializability. The protocol applied locks on data item, which may block (stop) accessing same data in other transactions.

The two-phase locking protocol divides the execution phase of the transaction into 3 parts.

- In the first part, when the transaction starts, it seeks permission for the lock it requires.
- In the second part, transaction acquires all the locks.

- In the third phase, the transaction cannot demand any new locks. It only releases the acquired locks.



2PL Protocol assumes that a transaction requires locks & unlocks being done in two phases.

- **Phase 1 -Growing Phase**
  - In this phase the transaction can onl
  -
  - y acquire locks, but cannot release any lock.
  - Ultimately the transaction reaches a point where all the lock it may need has been acquired. This point is called Lock Point.
- **Phase 2 -Shrinking Phase**
  - After Lock Point has been reached, the transaction enters the shrinking phase.
  - Transaction can only releasing all the acquired locks, but cannot acquire any new lock.

Initially a transaction is in the growing phase acquiring needed locks. After use transaction releases a lock and enters in the shrinking phase. It cannot issue more lock requests. Upgrading of lock is not possible in shrinking phase, but it is possible in growing phase.

Example- consider two transactions T1 and T2 as shown below:

T_1	T_2
lock-X(B);	lock-S(A);
read (B);	read (A)
b = b +10;	unlock (A);
write (B*);*	lock-S(B);
lock-X(A);	read (B);
read (A);	unlock (B);
a = a - 20;	display (a + *b);*
write (A);	
unlock (B);	
unlock (A);	

- In **T1** we put an exclusive-lock on **B** (data-item), so we can read or write **B**. again, we put an exclusive-lock on **A**, so we can read or write to **A**. Finally, we unlock both B and A in that order. **So, T1 is in two-phase.**
- But, **T2** is not in two-phase because **T2** puts a shared-lock on data-item **A**, so read operation can be performed. Then, it unlocks **A**. after shared-lock on **B** and reads it. Then, it unlocks **B**. Finally, **T2** tries to display the result. Because 2PL protocol means it should acquire all locks first and then release them. So, it is not in 2-phase.

It is not necessary that unlock operations should appear at the end of transaction only. Like in**T1** above, we can move unlock(B) instruction to just after lock-X(A) instruction and still hold the two phase locking property. **Such a point is called as a lock-point of T *i.e.*, a point in the schedule where T has obtained its final lock** (end of growing phase). So, we can

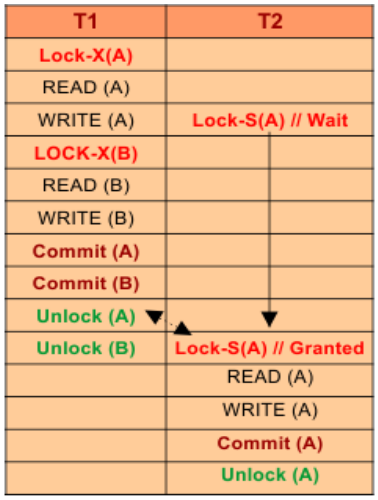
Advantages of 2PL protocol: To ensure Conflict Serializability

Disadvantages of 2PL protocol: Deadlocks and cascading roll-back may occur.

⊙ Variation of 2-Phase Locking Protocol

1. Strict 2-Phase Locking Protocol: A schedule will be in Strict 2PL if

- It must satisfy the basic 2-PL.
- Each transaction should holds all **Exclusive(X) Locks** until the Transaction is Commits or abort.



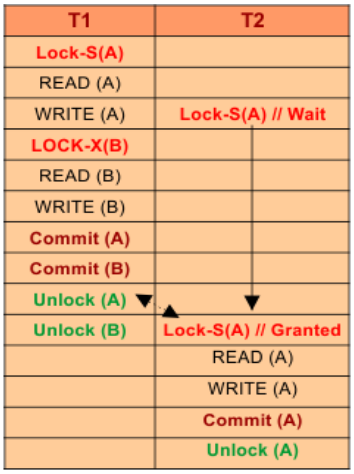
Strict 2-PL

2. Rigorous 2-Phase Locking Protocol

It is similar to Strict 2-PL in its advantage and disadvantage but little bit more strict than Strict 2-PL.

A schedule will be in Rigorous 2PL if

- It must satisfy the basic 2-PL.
- And each transaction should holds all **Exclusive(X) Locks** as well as **Shared(S) Locks** until the Transaction is Commits or abort.



Rigorous 2-PL

Difference between Strict and Rigorous 2PL

In Strict 2PL, If T1 acquire Shared lock on some data (Say A) and T2 also request for shared lock on same data ("A") then it is granted.

In Rigorous 2PL, If T1 acquire Shared lock on data "A" and T2 also request for shared lock on same data "A" then it will not be granted. Because T1 has to commit before to grant shared lock to T2. Following figure explain all

T1	T2
Lock-S(A)	
READ (A)	
WRITE (A)	Lock-S(A) // Granted
LOCK-X(B)	
READ (B)	
WRITE (B)	
Commit (A)	
Commit (B)	
Unlock (A)	
Unlock (B)	

Strict 2-PL

T1	T2
Lock-S(A)	
READ (A)	
WRITE (A)	Lock-S(A) // Wait
LOCK-X(B)	
READ (B)	
WRITE (B)	
Commit (A)	
Commit (B)	
Unlock (A)	
Unlock (B)	Lock-S(A) // Granted
	READ (A)
	WRITE (A)
	Commit (A)
	Unlock (A)

Rigorous 2-PL

Advantages of Strict/ Rigorous 2PL

- **Recoverable:** As T2 can't acquire any lock until T1 committed or abort. So if T1 fail then no effect on the T2 so schedule will become recoverable.
- **Cascadeless:** As T2 can't acquire any lock until T1 committed or abort. So if T1 fail then no effect on the T2 so schedule will become cascadless. Cascadless means rollback of one transaction doesn't affect the other transactions.

Note: Still **deadlock and starvation problems** are exist in strict 2-PL

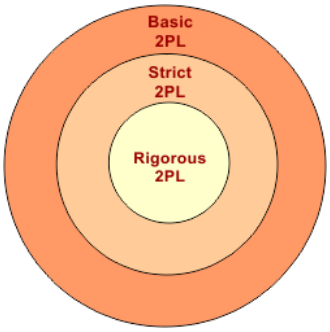
3. Conservative 2-Phase Locking Protocol

It is just a assuming technique which cannot implemented where,
Before to start the transaction, the transaction (i.e. T1) should hold all the locks on all data items(i.e. A, B, C etc). In this way rest of transactions (T2, T3 … Tn) will not access the data until T1 commit or abort.

Conservative 2PL prevents deadlocks along with recoverability and cascadless. Because when any transaction holds all the resources then it will never go to deadlock state. it is difficult to use in practice because of need to pre-declare the read-set and the write-set which is not possible in many situations.

Compression of 2PL Categories

Rigorous is more strict than strict 2PL and basic 2PL.



Timestamp based protocol

The timestamp-based algorithm determine serializability order of concurrent transactions based on their timestamps in a schedule. This protocol ensures that every conflicting read and write operations are executed in timestamp order. This protocol uses **System Time or Logical Counter** as a Timestamp. Timestamp of transaction denoted by TS (T1), timestamp values are assigned to every submitted transactions in the system and consider its start time of transaction. It uniquely identifies transaction in a schedule. Timestamp ordering does not use locks. Deadlocks cannot occur.

The timestamp of older transaction (T1) is less than the timestamp of a newly entered transaction (T2) i.e., TS(T1) < TS(T2).The priority of the older transaction is higher that's executes first.

Example: Suppose there are three transactions T1, T2, T3 and all are executing parallel on same data item ("A") as

- T1 arrive at time 8:00am So, assign as timestamp value as 100 called **Older**
- T2 arrive at time 8:10 am So, assign as timestamp value as 200 called **Younger**
- T3 arrive at time 8:15 am So, assign as timestamp value as 300 called **Youngest**

T1 has the higher priority, so transaction T1 which holds minimum timestamp value will execute first because it entered in the system first.

Timestamps can be generated by using,

i) System Clock: When a transaction enters in the system, its assigned a timestamp which is equal to time of system clock.

ii) Logical Counter: counter value is incremented each time for a new transaction entered.

Two timestamp values for every data item **A**

- **WTS(A) or (W-Timestamp(A)):** It represents highest timestamp value of the transaction that successfully executed the write operation on A.
- **RTS(A) or (R-Timestamp(A)):**It represents the highest timestamp value of the transaction that successfully executed the read operation on A.

If there are multiple data items in the schedule (i.e. A, B, C…) then each data item holds its own Read and Write timestamp as given below

- **RTS(A) = 300**
- **WTS(A) =200**
- **RTS(B) =100**
- **WTS(B) =100**

T1 (Timestamp =100)	T2 (Timestamp =200)	T3 (Timestamp =300)
R(A)		R(B)
	R(A)	W(B)
R(B)	W(A)	R(A)
W(B)		

➢ **Basic Time-stamp Ordering (TO):**

Whenever some transaction T tries to issue a R(A) or W(A) operation, the basic time-stamp ordering algorithm compares the time-stamp(T)value of read_TS(A) and write_TS(A).

Timestamp Ordering Protocol

The Time stamp ordering protocol ensures that any conflicting read and write operations are executed in time stamp order. It determines the serializability order of transactions.

Basic Timestamp ordering protocol works as follows:

1. Suppose that transaction Ti issues read (A):

- If WTS(A) > TS (Ti), then Ti Rollback
- Else (otherwise) execute R(A) operation and **SET** RTS (A) = MAX {RTS(A), TS(Ti)}

2. Suppose that transaction Ti issues write (A):

- If RTS(A) > TS (Ti), then Ti Rollback
- If WTS(A) > TS (Ti), then Ti Rollback
- Else (otherwise) execute W(A) operation and **SET** WTS (A) = TS(Ti)

Example of Timestamp ordering Protocol

Time of Transaction	T1 (Timestamp = 100)	T2 (Timestamp = 200)	T3 (Timestamp = 300)
Time 1	R (A)		
Time 2		R (B)	
Time 3	W (C)		
Time 4			R (B)
Time 5	R (C)		
Time 6		W (B)	
Time 7			W (A)

Solution: In above table A,B,C are data values. Read and Write timestamp values are given "0".

—	A	B	C
RTS	0	0	0
WTS	0	0	0

At time 1, transaction T1 wants to perform read operation on data "A" then according to **Rule 01,**

- WTS(A) > TS(T1) = 0>100 **// condition false**
- Go to else part and **SET** RTS(A)= MAX {RTS(A), TS(T1)}
- So, RTS (A) = MAX{0,100}= 100.
- So, finally RTS(A) is updated with 100 Updated table,

—	A	B	C
RTS	**100**	0	0
WTS	0	0	0

At time 2, transaction T2 wants to perform read operation on data **"B"** then according to **Rule 01,**

- WTS(B) > TS(T2) = 0>200 // condition false
- Go to else part and SET RTS (B) = MAX {RTS(B), TS(T2)} So,
- RTS (B) = MAX{0,200} = 200.
- So, finally RTS(B) is updated with 200 Updated table,

—	A	B	C
RTS	**100**	**200**	0
WTS	0	0	0

At time 3, transaction T1 wants to perform write operation on data **"C"** then according to **Rule 02,**

- RTS(C) > TS(T1) = 0>100 // condition false
- Go to second condition, WTS(C) > TS(T1) = 0>100 // again condition false
- Go to else part and SET WTS (C) = TS(T1) So,
- WTS (C) = TS(T1) = 100.
- So, finally WTS(C) is updated with 100 Updated table,

—	A	B	C
RTS	**100**	**200**	0
WTS	0	0	**100**

At time 4, transaction T3 wants to perform read operation on data **"B"** then according to **Rule 01,**

- WTS(B) > TS(T3) = 0>300 // condition false
- Go to else part and SET RTS (B) = MAX {RTS(B), TS(T3)} So,
- RTS (B) = MAX{200,300} = 300.
- So, finally RTS(B) replace 200 and updated with 300 Updated table,

—	A	B	C
RTS	**100**	**200**	0
WTS	0	0	**100**

At time 5, transaction T1 wants to perform read operation on **data "C"** then according to **Rule 01,**

- WTS(C) > TS(T1) = 100>100 // condition false
- Go to else part and SET RTS (C) = MAX {RTS(C), TS(T1)} So,
- RTS (A) = MAX{0,100}= 100.
- So, finally RTS(C) is updated with 100 Updated table,

—	A	B	C
RTS	**100**	**300**	**100**
WTS	0	0	**100**

At time 6, transaction T2 wants to perform write operation on data **"B"** then according to **Rule 02,**

- RTS(B) > TS(T2) = 300>200 // condition True
- **According to Rule 2:** if condition true then Rollback T2.

When T2 rollback then it never be resume, it will restart with new timestamp value. T2 restart after completion of all running transactions, so in this example T2 will restart after completion of T3.
It happens due to conflict where the older transaction (T2) want to perform write operation on data "B" but younger transaction (T3) already Read the same data "B". Table will remain the same

—	A	B	C
RTS	100	300	100
WTS	0	0	100

At time 7, transaction T3 wants to perform write operation on data **"A"** then according to **Rule 02,**

- RTS(A) > TS(T3) = 100>300 // condition false
- Go to second condition, WTS(A) > TS(T3) = 100>300 // again condition false
- Go to else part and SET WTS (A) = TS(T3) So, WTS (A) = 300.
- So, finally WTS(A) is updated with 300 Updated table,

—	A	B	C
RTS	100	300	100
WTS	300	0	100

➢ **Advantages of Timestamp Protocol**

Timestamp protocol always ensure Serilizabilty and Deadlock removal because Transaction with smaller timestamp (TS) will came first in execution sequence than Transaction with higher TS.

➢ **Disadvantages of Timestamp Protocol**

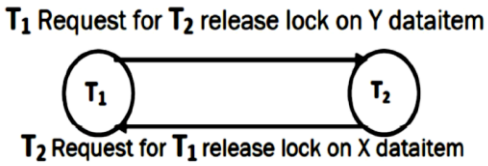
- Schedule may not be recoverable.
- Schedule may not be cascading free.

Deadlock

Deadlock: "A system is in a deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set."

There exists a set of waiting transactions (T0,T1, Tn) such that T0 is waiting for a data item that is held by T1 and T1 is waiting for a data item that is held by T2 so on.
If cycle is exists in graph then deadlock state occurs. If cycle does not exist in graph then deadlock state not occurs.
A deadlock is a condition in which two or more transaction is waiting for each other deadlock (T1 and T2). An example is given below

T_1	T_2
Lock-X (A); Write(A); Wait for **lock-X** on B	 Lock-X(B); Write(B); Wait for **lock-X** on A



Example: A traffic, which is going in one direction. Here, a bridge is considered a resource. So, when Deadlock happens, it can be easily resolved if one car backs up (Preempt resources and rollback). Several cars may have to be backed up if deadlock situation occurs. So starvation is possible.

Deadlock handling methods

- Deadlock Avoidance
- Deadlock prevention
- Deadlock detection

➢ **Deadlock Avoidance:** When a database is stuck in a deadlock state, then it is better to avoid the database rather than aborting or restating the database. This is a waste of time and resource.

Deadlock avoidance mechanism is used to detect any deadlock situation in advance. A method like "wait for graph" is used for detecting the deadlock situation but this method is suitable only for the smaller database. For the larger database, deadlock prevention method can be used.

Another method for avoiding deadlock is to apply both row level locking mechanism and READ COMMITTED isolation level. However, it does not guarantee to remove deadlocks completely.

➢ **Deadlock Prevention:**

We can prevent deadlocks by giving priority to each transaction and ensuring that lower priority transactions are not allowed to wait for higher priority transactions (or vice versa).

One way to assign priorities is to give each transaction a timestamp when it starts up. The lower timestamp, the higher transaction priority that is, the oldest transaction has the highest priority.

If a transaction **Ti** requests a lock and transaction **Tj** holds a conflicting lock, the lock manager can use one of the following two policies:

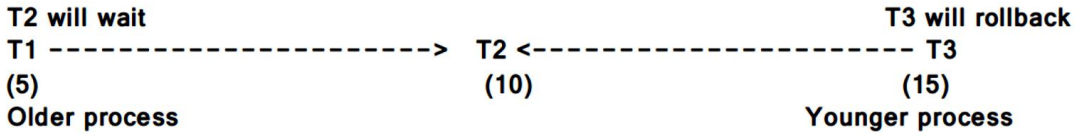
- Wait-die
- Wound-wait

Condition	Wait-die	Wound-wait
Older process need a resource which is held by Younger process	Older process wait	Younger process Die
Younger process need a resource which is held by Older process	Younger process Die	Younger process Die

⊙ **Wait-die**

In the wait-die approach, if a transaction requests a resource which is locked by another transaction, then DBMS simply checks the timestamp of both transactions. if requesting transaction is older than the transaction which is holding lock on data item, the requesting transaction is allowed to wait. But if requesting transaction is younger than the transaction that is holding the lock, then the requesting transaction is aborted and rolled back.

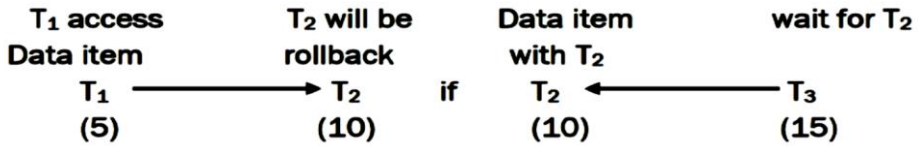
Consider following transactions with timestamp values T1 (5), T2 (10) and T3 (15). If request for lock on data item which is locked by T2 then, T1 has allowed waiting. If T3 request for lock on data item which is locked by T2 then T3 aborted and rolled back.



The wait-die scheme is non-preemptive scheme because only a transaction requesting a lock can be aborted. As a transaction produces older (and its priority increases), it tends to wait for more and more younger transactions.

⊙ **Wound-wait**

In wound-wait approach, if the requesting transaction is older than the transaction which is holding lock on data item, the younger transaction is aborted and older transaction apply lock on data item. If the requesting transaction is younger then the transaction which is holding lock on data item the requesting transaction is allowed to wait. Consider following transactions with timestamp values T1 (5), T2 (10) and T3 (15). If T1 request for lock on data item which is locked by T2 then, T2 is aborted and T1 apply lock on data item. If T3 request for lock on data item which is locked by T2 then T3 is wait for data item.



This scheme is based on a preemptive technique and is a complement to the wait-die scheme. In either case no deadlock cycle can develop.

Wait – Die	Wound -Wait
It is based on a non-preemptive technique.	It is based on a preemptive technique.
In this, older transactions must wait for the younger one to release its data items.	In this, older transactions never wait for younger transactions.

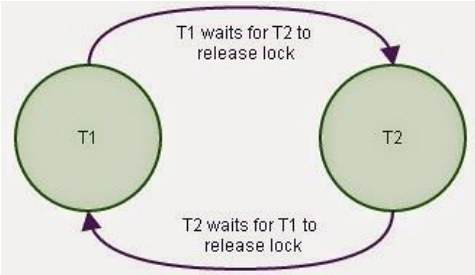
The number of aborts and rollback is higher in these techniques.	In this, the number of aborts and rollback is lesser.

➢ **Deadlock Detection:**

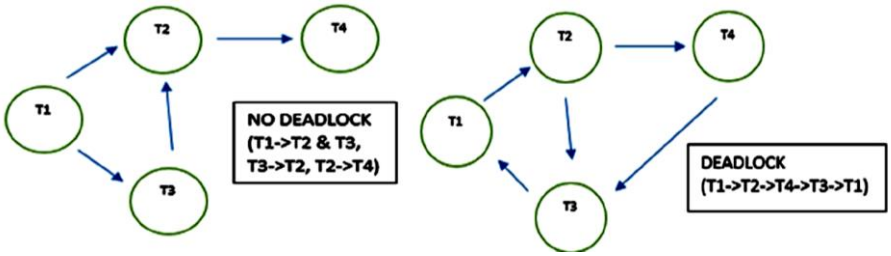
When a transaction waits indefinitely to obtain a lock, then the DBMS should detect whether the transaction is involved in a deadlock or not. The lock manager maintains a Wait for graph to detect the deadlock cycle in the database.If a cycle exists, then one transaction involved in the cycle is selected (victim) and rolled back.

Deadlocks can be described as a *wait-for* graph **G = (V, E)**

- Set of vertices (V)– the set of transactions currently going on in a Database, like T1, T2, … Tn.
- Set of edges (E) – transaction Ti is waiting for a resource held by transaction Tj. i.e. E is a set of directed edges **(Ti ->Tj)**



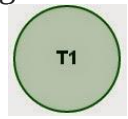
- When **Ti** requests a data item currently being held by **Tj**, then the directed edge **Ti -->Tj** is inserted in the wait-for graph. This edge is removed only when **Tj** is no longer holding a data item needed by **Ti**.
- The system is in a deadlock state if and only if the wait-for graph has a cycle. Must invoke a deadlock-detection algorithm periodically to look for cycles.



Example: Assume that there are 4 transactions as T1, T2, T3, and T4 that are going on in a system at a particular point in time. The data items and the lock modes needed for the data items for all the transactions are given in table 1 below;

Transactions	Data items requested	Lock mode
T1	Q	Shared lock
T2	P	Exclusive lock
	Q	Exclusive lock
T3	Q	Shared lock
T4	P	Exclusive lock

Stage 1: Assume that T1 has acquired lock on Q and being executed. The wait-for graph contains no edges. It contains only transaction T1 as given in the figure.



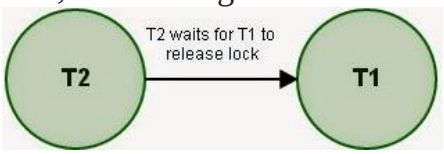
T1 locked a data item Q

Stage 2: Now, transaction T2 started and requesting for the data items P and Q in write mode. The lock manager can grant lock on P, but cannot grant lock on Q as Q is already held by T1 and the *held lock* and *requested lock* are incompatible. At this stage, a new edge is inserted like T2 → T1 into the Wait-for graph and the graph looks like the figure given below;

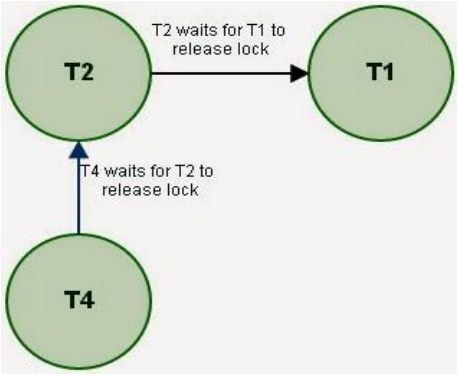
edge (T2,T1) is inserted

Stage 3: Now, transaction T3 requests for data item Q. The lock manager can grant lock on Q to transaction T3. The reason is, T1 locked Q in read mode and T3 requests in read mode, and they are compatible. Hence, the lock is granted. So, no new edge is included in the Wait-for graph (Figure).

no changes

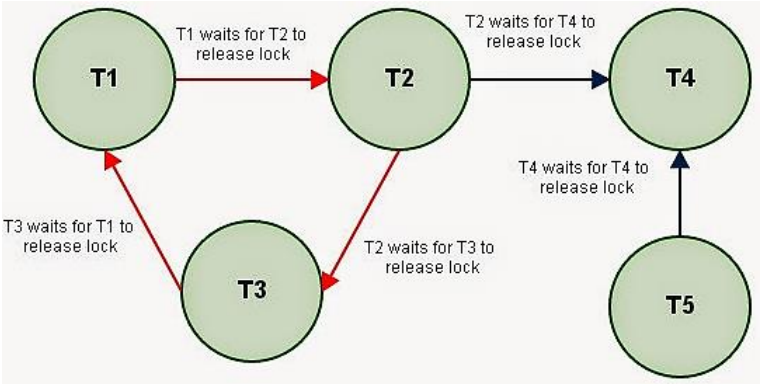


Stage 4: Next, transaction T4 requests data item P in write mode. Again, P is held by T2 and in incompatible mode. So, the lock manager cannot grant. Hence, a new edge T4 → T2 is included in the graph. Now, the wait-for graph looks like the one given in the figure 5 below;



new edge (T4,T2) is inserted

If we get any more requests, the graph will produce this way, if any transactions releases the lock held by them, then the wait-for edge will be removed. This deadlock situation need not involve all the transactions. It may involve some of the transactions.



Wait-for-graph with both deadlocked transactions and free transactions

Only the transactions T1, T2 and T3 are formed a cycle, thereby entered a deadlock state. Transactions T4 and T5 are not part of the current deadlock state.

- **Deadlock Recovery**

After detecting the deadlock, a system has to take is to recover the system from the deadlock state. This can be achieved through rolling back one or more transactions. But it is difficult part to choose one or more transactions as victims.

Recovery from deadlock can be done in three steps;

1. Selection of victim: Given a set of deadlocked transactions, we have to identify transactions that are to be rolled-back for successful break deadlock state. This is done by identifying transactions that are cost minimum.

Guidelines to choose victim:

- Length of the transaction– We need to choose the transaction which is younger.
- Data items used by the transaction– The transactions that are used less number of data items.
- Data items that are to be locked – The transaction that needs to lock many more data items

compared to that are already locked.
- How many transactions to be rolled back? – We need to choose transactions that would cause less number of other transactions to be rolled back (cascading rollback)

2. Rollback: Once we have identified the transaction that are to be rolled back, then rollback them. This can be done in two ways-
- Full rollback –abort the transaction and restart it.
- Partial rollback –if the system maintains additional information like the order of lock requests, save points etc.

3. Starvation: to avoid a transaction to be chosen repeatedly as victims for rollback.